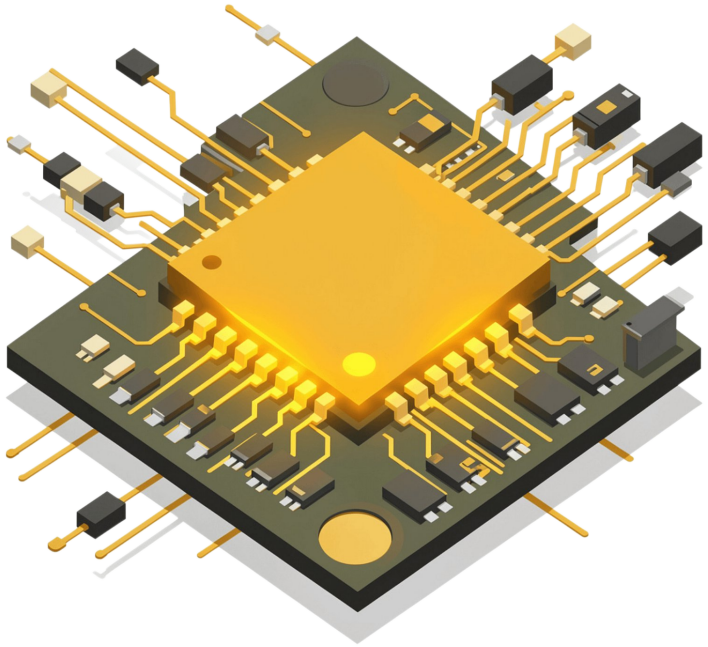


NeuroSwap: Proactive User-Space Memory Orchestration for LLM Inference on Constrained Edge Devices

BCSE303P Operating Systems Lab Project Review 1

Literature Review: The Edge AI Bottleneck

The Trend

Massive industry shift towards local "Edge AI" and virtualized inference (WSL, Docker).

The Hardware Constraint

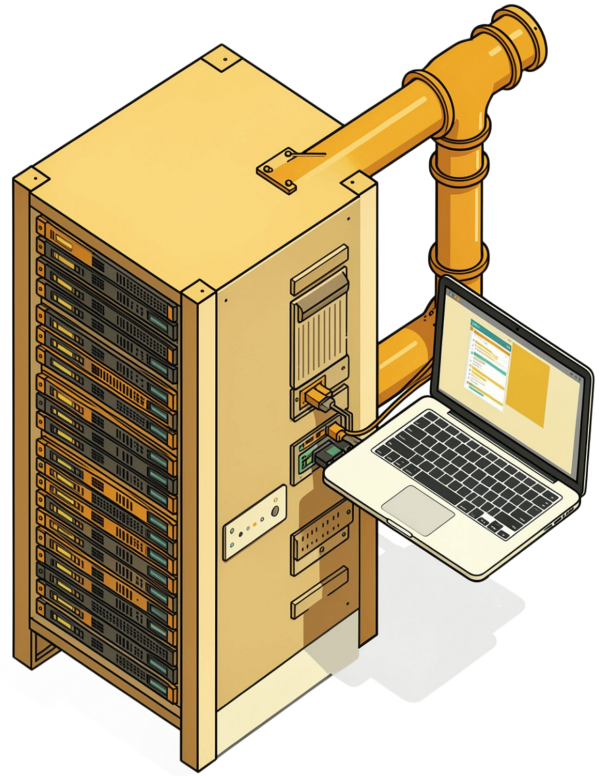
Generative AI models (like LLaMA-3) require gigabytes of RAM, heavily bottlenecking edge devices.

Key Literature

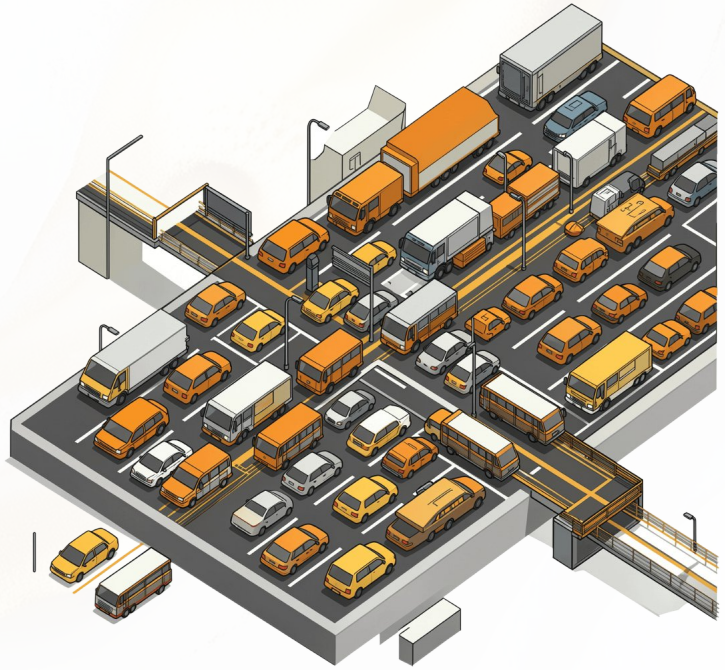
Apple's "LLM in a flash" (2024) proved SSDs can supplement RAM. Current 2025/2026 systems research identifies "KV-Cache Residency" as the ultimate bottleneck for sequential token generation.

The Research Gap

Existing papers focus on GPU offloading. There is a critical lack of research on optimizing the Host-OS paging mechanisms for CPU-bound virtual environments.



Problem Statement: The LRU Pathology



1 The Core Flaw

The default Linux Virtual Memory manager relies on the LRU (Least Recently Used) algorithm.

2 Blind to AI

Linux has no concept of "Neural Network Layers" or the AI's "KV-Cache" (context history).

3 The Thrashing Disaster

Under heavy memory pressure, Linux blindly swaps the critical KV-Cache to the slow SSD to make room for incoming model weights.

4 The Result

Massive "Major Page Faults." The CPU halts, waiting for the SSD, causing AI token-generation latency to spike by up to **50x**.

Project Objectives

Quantify the Baseline

Use the Linux `perf` kernel tool to benchmark exact page-fault overhead during LLM inference in WSL.

Develop NeuroSwap

Build a user-space C++ memory orchestrator that intercepts AI memory allocations.

Application-Aware Paging

Use `mlock()` to permanently pin the KV-Cache in physical RAM. Use `madvise()` for asynchronous, layer-by-layer prefetching and instant eviction.

Evaluate & Publish

Reduce Major Page Faults by >80% and submit findings to a Tier-1 IEEE/ACM systems conference.

Integrating ML for Systems: Dynamic Prefetching

The Limitation of Static Paging

Hardcoded memory prefetching fails when hardware performance fluctuates (e.g., SSD thermal throttling or background OS tasks).

Our ML Integration

Embedding a lightweight Linear Regression model directly inside the C++ memory orchestrator.

Dynamic Telemetry

The model continuously monitors real-time CPU matrix-compute times and SSD block-I/O read latencies.

Self-Tuning Architecture

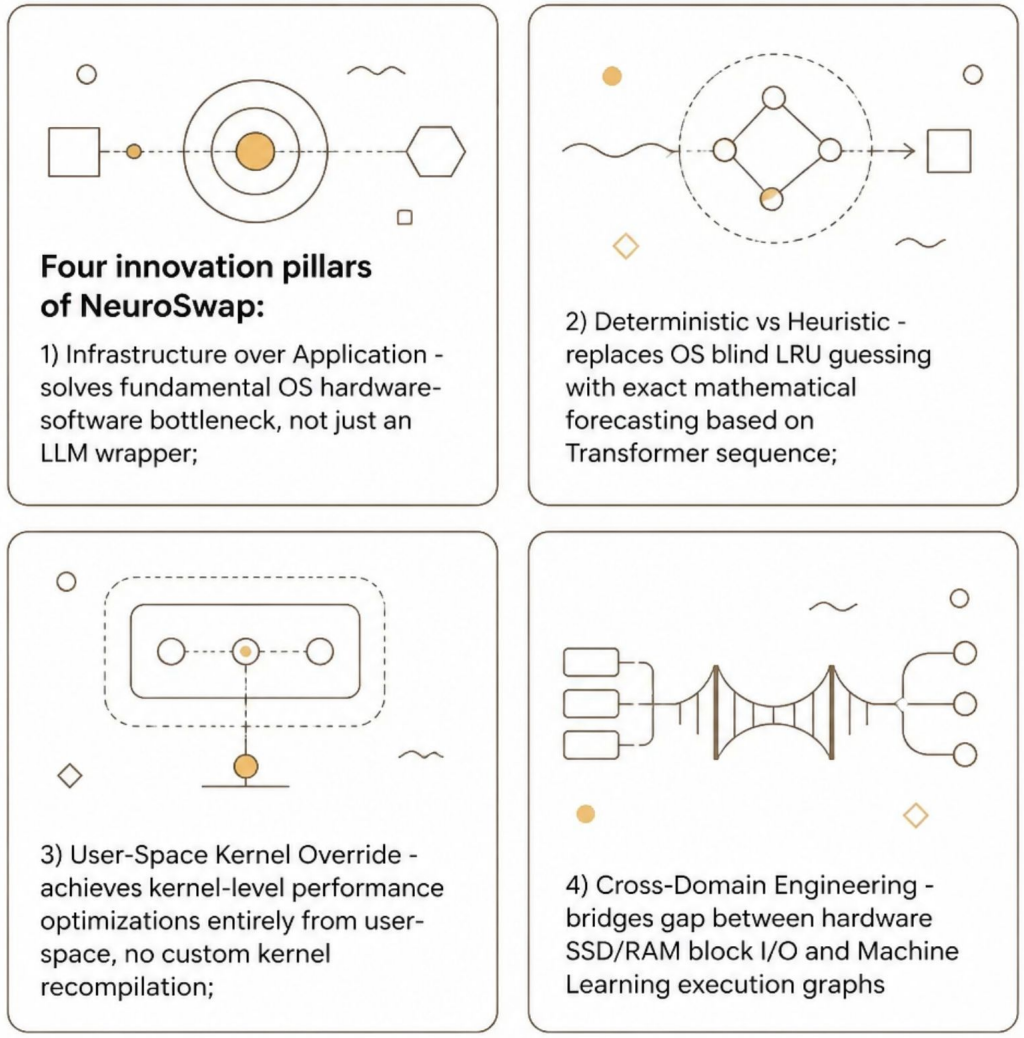
The ML agent dynamically adjusts the execution timing of the madvise prefetch commands, adapting to hardware lag on the fly to ensure zero CPU idle time.

Existing Solutions vs. NeuroSwap

A critical comparison of approaches to the edge LLM memory problem — only one eliminates thrashing without compromise.

Solution	Approach	Critical Flaw
Model Quantization (GGUF)	Shrinks the AI to fit in RAM	Degrades the AI's intelligence
Standard OS Paging (LRU)	Lets Linux manage memory automatically	Blindly swaps to SSD, causing 50x latency spikes
Hardware Upgrades	Buy machines with massive Unified Memory	Too expensive, destroys Edge AI accessibility
NeuroSwap	**Mathematically predicts memory needs, pins KV-Cache, bypasses OS thrashing**	**Keeps model intelligent, costs $0, eliminates thrashing**

Novelty & Innovation



Infrastructure over Application

Not just another LLM wrapper; this solves a fundamental OS hardware-software bottleneck.

Deterministic vs. Heuristic

Replaces the OS's blind guessing (LRU) with exact mathematical forecasting based on the Transformer sequence.

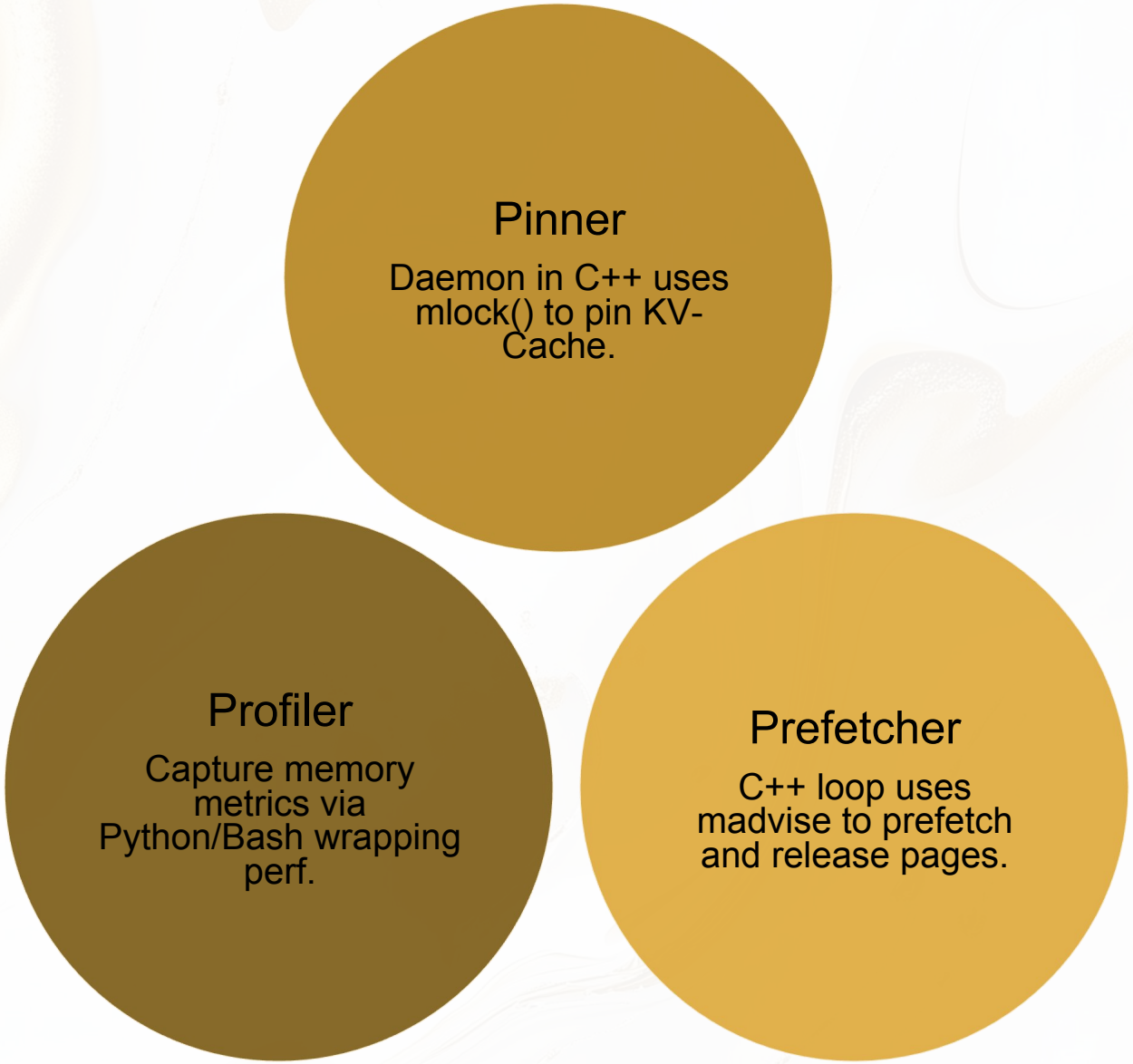
User-Space Kernel Override

Achieves kernel-level performance optimizations entirely from user-space, requiring no custom kernel recompilation.

Cross-Domain Engineering

Bridges the critical gap between hardware (SSD/RAM block I/O) and Machine Learning execution graphs.

Next Steps: System Architecture



NeuroSwap's three-module pipeline forms a closed-loop memory orchestration system: the Profiler captures ground-truth metrics, the Pinner guarantees KV-Cache residency, and the Prefetcher eliminates stall latency through asynchronous layer-by-layer staging.